

04.06.2024

Übungsblatt 6

Hinweis: Bearbeiten Sie das Übungsblatt als 4er-Gruppe.

Speichern Sie Ihre Lösungen bis spätestens **11.06.2024, 18:00 Uhr** auf Ihrem

Gruppen-Übungsserver `scilab-00???.cs.uni-kl.de`

im Ordner `/home/lamp/htdocs/ex06` im **HTML-Format** ab.

Die Abgabe erfolgt indem Sie zusätzlich die **Lösungsdateien (als ZIP-File) auf OLAT hochladen**.

Bitte geben Sie bei jeder Abgabe mit an **welche Gruppenmitglieder*innen aktiv an der Bearbeitung des aktuellen Übungsblattes mitgewirkt haben**. Fügen Sie dazu dem auf OLAT hochzuladenen ZIP-File eine zusätzliche Datei "TeamBlatt6.txt" mit den Namen der aktiven Gruppenmitglieder*innen hinzu.

Falls Sie Fragen haben nutzen Sie bitte das Forum in OLAT oder schicken Sie eine Mail direkt an w2t@cs.uni-kl.de.

Viel Erfolg!

1. Aufgabe: GET oder POST

(3 P)

Für welche der folgenden Aufgabe würden Sie GET oder POST benutzen?

Geben Sie jeweils auch eine sinnvolle „sprechende“ URL für den Request an.

- a) Dessert-Rezept „Schoko-Muffins mit Käse“ anzeigen
- b) Mail #17 als gelesen markieren
- c) Warenkorb bestellen
- d) User „Joe“ einloggen (Begründung für Ihre Wahl!)
- e) User ausloggen (Begründung für Ihre Wahl!)

2. Aufgabe: Dynamische-Webseite analysieren (6 P)

Speichern Sie als User `lamp` auf `scilab-00???.cs.uni-kl.de` den folgenden Quellcode in eine Datei mit dem Namen `B06A2.php` im Verzeichnis `/home/lamp/htdocs/`.

```
<?php
    $count = @$_GET['count'];
    $url="?count=" . ($count+1);
?>
<!DOCTYPE html>
<html>
<head>
    <title>Counter (<?php echo $count; ?>)</title>
    <meta http-equiv=refresh content="2; URL=<?php echo $url ?>">
</head>
<body>
    <h1>Count: <?php echo $count; ?></h1>
    <?php echo date('s'); ?>
</body>
</html>
```

Rufen Sie nun im Browser die Webseite `http://scilab-00???.cs.uni-kl.de/B06A2.php` ab und beantworten Sie folgenden Fragen:

- Was tut diese Webseite?
- Warum funktioniert das bei Aufruf ohne GET-Parameter (z.B. beim ersten Aufruf)?
(Beschreiben Sie die Punkte, an denen sich das im PHP-Code auswirkt.)
- Ist dieses Konstrukt sicher gegen bösartige Parameter?
Was sind hier kritische Stellen im Code?
Können Sie eine URL angeben, über deren Aufruf eine fehlerhafte Seite entsteht, die vom HTML-Validierer nicht mehr akzeptiert wird?
- Können Sie die Seite so modifizieren, dass sie sicher gegen bösartige Parameter ist?

3. Aufgabe: Formulare

(1+3+2+0.5+0.5+1+1 = 9 P)

Sie sollen eine Wetterbeobachtungs-Webseite einrichten, in der man zu einem Ort und Datum einen Wetterbericht eingeben kann.

Lösen Sie folgende Aufgaben in einem GET-Formular in der Datei **B6A3.php**. Ordnen Sie die Felder in einer Tabelle an. Betiteln Sie die Felder so, dass ein normaler Benutzer ihren Sinn verstehen kann.

a) Auswahlfelder (statisch)

Statt eines freien Eingabefelds kann man in Formularen auch fest vorgegebene Optionen zur Auswahl anbieten. Erzeugen Sie manuell ein Eingabefeld mit dem Namen „ort“, in dem man zwischen den Optionen „New York“, „Tokio“, „London“ und „Kaiserslautern“ wählen kann. (siehe <https://wiki.selfhtml.org/wiki/HTML/Tutorials/Formulare/Auswahllisten>)

b) Auswahlfelder (generiert)

Erzeugen Sie Per PHP ein Eingabefeld mit dem Namen „datum“, in dem man die Werte „1.1.“ ... „30.1.“, „1.2.“ ... „30.2.“ usw. bis „30.12.“ wählen kann.
(Wir nehmen an, jeder Monat hat genau 30 Tage.)

c) Auswahlfelder (hierarchisch)

Die Liste ist offensichtlich etwas unübersichtlich geworden. Fügen Sie nun Monats-Gruppen in diese Liste ein, damit man zweistufig (Monat → Tag) in der Liste wählen kann. (Siehe https://wiki.selfhtml.org/wiki/HTML/Tutorials/Formulare/Auswahllisten#Verschachtelte_Auswahllisten_.28Menuestruktur.29_definieren)

d) Auswahlfelder (Vorauswahl)

Vorausgewählt soll der 7.3. sein.

e) Radiobuttons

Fügen sie Radio-Buttons hinzu, mit denen man den Wert „wetter“ zwischen den Optionen „Sonne“, „Regen“ und „Schnee“ auswählen kann. (Siehe https://wiki.selfhtml.org/wiki/HTML/Tutorials/Formulare/Radio-Buttons_und_Checkboxen)

f) Checkbox

Fügen Sie die Checkboxen „windig“ und „neblig“ und „langweilig“ hinzu.

Windig soll vorab selektiert sein.

Geben Sie die vom Formular erhaltenen GET-Parameter z.B. mit folgendem Fragment aus:

```
<pre><?php print_r($_GET); ?></pre>
```

4. Aufgabe: Formulare (HTML5)

(2+2+2=6 P)

Bart Simpson[®] hat des öfteren Strafarbeiten zu erledigen. Helfen Sie Ihm indem Sie in der Datei **B6A4.php** im Verzeichnis `/home/lamp/htdocs/` folgendes implementieren:

- a) In einem POST-Formular soll der Strafarbeiten-Text und die Anzahl der Wiederholungen angegeben werden. Schickt man das Formular ab wird die Strafarbeit entsprechend generiert. Seine Lehrerin verlangt meistens 42 Wiederholungen des Satzes. Gestalten Sie die Ergebnisseite optisch so, dass es aussieht als ob mit weißer Kreide an einer grünen Tafel mit braunem Rand geschrieben wird.

- b) Formular-Validieren mit PHP:

Folgende Bedingungen sollen eingehalten werden:

- mindestens 2 und maximal 100 Wiederholungen
- mindestens 5 und maximal 256 Zeichen für den Strafarbeiten-Text

Wird eine oder mehrere Bedingungen verletzt, geben Sie an dem betroffenen Formular-Elementen eine passende mit CSS hervorgehobene Hinweis-Meldung aus. Erst wenn alle Bedingungen erfüllt sind wird die Strafarbeit generiert.

- c) Formular-Validieren mit HTML 5 (im Client):

Nutzen Sie nun Formular-Elemente aus HTML5 mit denen die Validierung bereits im Browser stattfinden kann bzw. eine nicht erlaubte Eingabe erst gar nicht möglich ist. (siehe http://www.w3.org/wiki/HTML5_form_additions)

Wird die serverseitige Überprüfung der Eingabedaten überhaupt noch benötigt?

5. Aufgabe: PHP-Postback

(6 P)

Implementieren Sie in der Datei **B6A5.php** ein Registrierungs-Formular mit dem in der Vorlesung vorgestellten Postback-Verfahren. Folgende Bedingungen müssen erfüllt sein:

- Benutzername muss eine Universitäts-Email-Adresse („rptu.de“) sein.
- Passwort muss mindestens 12 Zeichen und neben Buchstaben auch Zahlen und Sonderzeichen enthalten.

Sind die Eingaben korrekt, dann geben Sie unterhalb des Formulars den Benutzernamen und dahinter das gehashte Passwort aus.

- Warum ist die Funktion „base64_encode“ dazu ungeeignet?
- Wieso sollten Sie zusätzlich einen Salt verwenden?